MECHATRONICS AND IOT ASSIGNMENT
REPORT - 10

IIoT-Based HVAC Temperature and Humidity Monitoring System

PROJECT REPORT

KOWSHIK K 24MER025

NITHISH J 24MER037

KAVIN KRISHNA D S 24MER024

LOGESH S 24MER027

1. Project Overview:

HVAC (Heating, Ventilation, and Air Conditioning) systems are widely used in residential, commercial, and industrial environments to maintain comfortable indoor conditions. The efficiency and performance of an HVAC system depend heavily on maintaining appropriate temperature and humidity levels. Traditional monitoring methods often involve manual inspection and periodic observation, which may not provide continuous information about changing environmental conditions. As a result, fluctuations in temperature and humidity may go unnoticed, leading to reduced system performance, increased energy consumption, and discomfort for occupants.

To address these challenges, an IoT-Based HVAC Temperature and Humidity Monitoring System is proposed. The system continuously measures environmental parameters and transmits the collected data to a cloud platform through Wi-Fi. This enables real-time monitoring, remote access to environmental data, and analysis of historical trends. By providing continuous visibility into temperature and humidity conditions, the system helps improve HVAC performance, supports energy-efficient operation, and enables early detection of abnormal environmental conditions that may affect system reliability and indoor comfort.

2. Project Statement:

Maintaining optimal temperature and humidity levels within HVAC-controlled environments is essential for ensuring operational efficiency, indoor air quality, and equipment reliability. Traditional manual checks do not offer real-time tracking and can easily overlook sudden environmental fluctuations, leading to inefficiency and potential equipment strain.

Therefore, an IIoT-Based HVAC Temperature and Humidity Monitoring System is proposed to continuously track environmental parameters and transmit data to a cloud platform for real-time analysis.

The system is designed to:

- Continuously measure ambient temperature using the DHT11 sensor.
- Continuously measure relative humidity levels.
- Display real-time parameters on a local I2C LCD screen.

- Trigger local visual and audible alerts during abnormal thermal conditions.
- Transmit telemetry data wirelessly over Wi-Fi via MQTT.
- Store and log data on a cloud platform for historical trend visualization.
- Enable remote monitoring through an interactive cloud dashboard.

This system enables operators to monitor HVAC conditions remotely, identify thermal anomalies early, and maintain stable environmental conditions.

3. Proposed Solution:

The proposed IIoT-Based HVAC Temperature and Humidity Monitoring System uses a DHT11 temperature and humidity sensor paired with an ESP8266 NodeMCU to continuously track environmental parameters in real time. The sensor measures ambient temperature and relative humidity, while the micro-controller processes these readings and displays them on a local 16×2 I2C LCD screen. The collected telemetry is transmitted wirelessly via Wi-Fi to a cloud platform (Adafruit IO) using the MQTT protocol, enabling continuous remote monitoring.

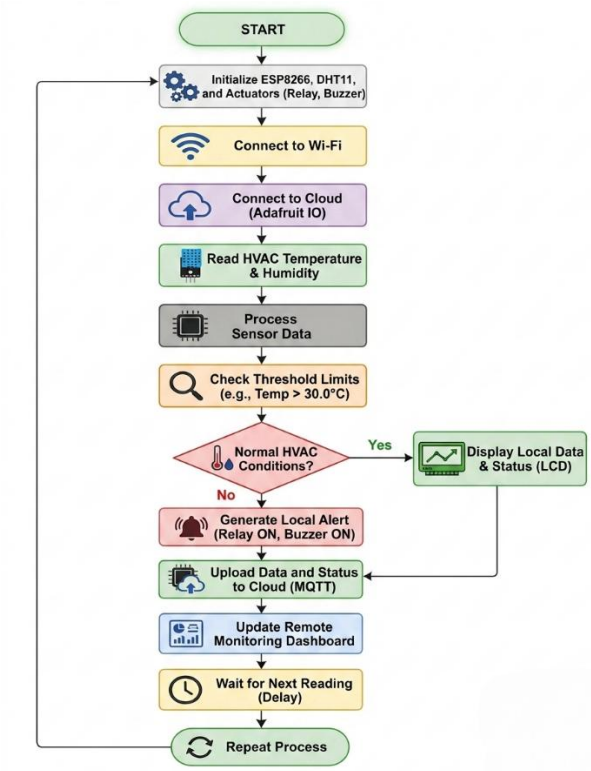
If environmental conditions exceed pre-set thresholds—such as a temperature rise above $30.0°C$—the system triggers local alerts using a piezo buzzer and an active-low relay module to signal high-temperature states. Real-time metrics and system statuses are uploaded to a cloud dashboard for historical trend analysis.

The system operates through the following steps:

- Temperature Monitoring: Continuously reads ambient temperature using the DHT11 sensor at regular intervals.
- Humidity Monitoring: Measures real-time relative humidity inside the monitored space.
- Local Display: Outputs live temperature, humidity, and status messages to a 16×2 I2C LCD.
- Threshold & Alert Control: Evaluates sensor values against defined parameters and energizes the relay and buzzer when temperatures exceed limits ($> 30.0°C$).
- Wireless Telemetry: Transmits sensor readings and system statuses to the cloud platform via Wi-Fi using MQTT.

- Cloud Logging & Dashboard: Logs historical data and visualizes parameter trends on an interactive remote dashboard.

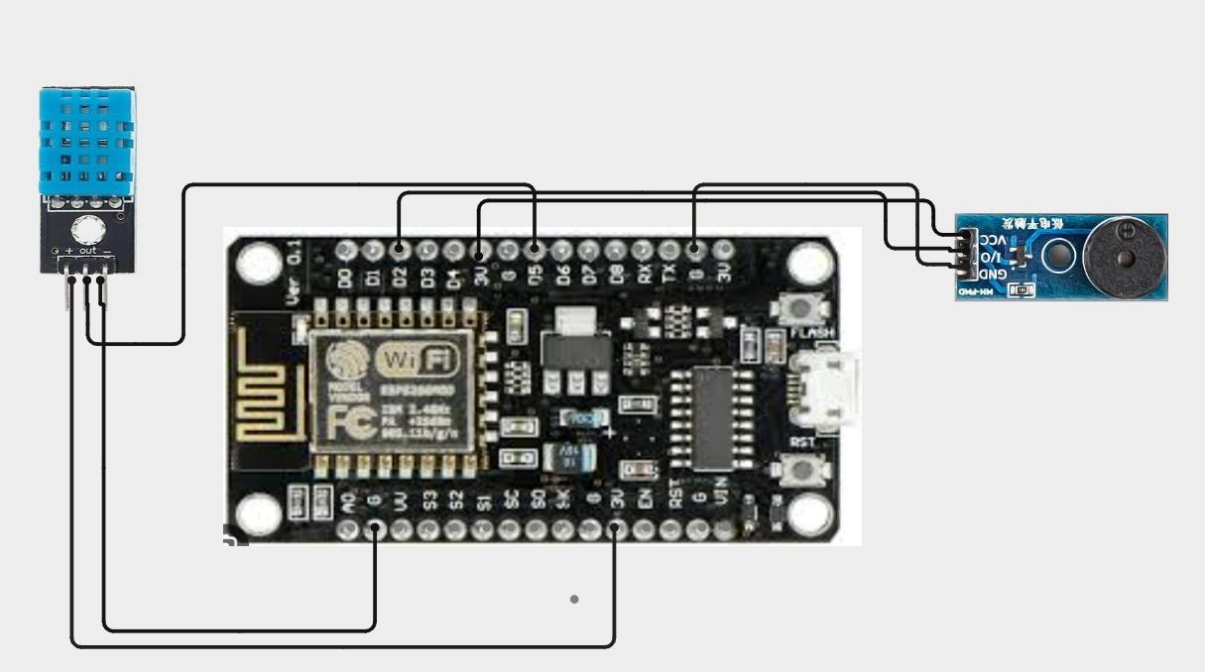
4. System Architecture:



5. Circuit Table:

Component	Connection	Quantity
DHT11 Temperature & Humidity Sensor	Data → D15 (GPIO15)	1 DOCX
16 × 2 I2C LCD Display	SDA → D21 (GPIO21), SCL → D22 (GPIO22)	1 DOCX
Active-Low Relay Module	IN → D26 (GPIO26)	1 DOCX
Buzzer Module	Signal → D25 (GPIO25)	1 DOCX
ESP8266 NodeMCU	3.3V / 5V & GND → All Components	1 DOCX
Breadboard & Jumper Wires	Interconnections	1 Set DOCX
USB Cable / Power Supply	5V Power Input → Micro-USB	1 DOCX

6. Circuit Design:



7. Algorithm:

Step 1: Start the system.
Step 2: Initialize the ESP8266 NodeMCU, DHT11 sensor
Step 3: Connect the ESP8266 to the configured Wi-Fi network.
Step 4: Establish communication with the cloud platform.
Step 5: Read temperature from the DHT11 sensor.
Step 6: Read humidity from the DHT11 sensor.
Step 7: Display the measured values on the OLED/LCD display.
Step 8: Compare the readings with predefined threshold values.
Step 9: If abnormal conditions are detected, activate the buzzer and relay.
Step 10: Upload temperature and humidity data to the cloud platform.
Step 11: Enable remote monitoring through the dashboard.
Step 12: Repeat the process at regular intervals.

8. Coding:

```
#include <ESP8266WiFi.h>
```

```
#include <ThingSpeak.h>
#include <DHT.h>

//--------------- WiFi -----------------
const char* WIFI_SSID = "Luky003";
const char* WIFI_PASS = "kowshik03";

//----------- ThingSpeak ---------------
unsigned long CHANNEL_ID = 3459486;
const char* WRITE_API_KEY = "EHDR6D4T2FDB8LHZ";

WiFiClient client;

//------------ DHT11 -------------------
#define DHTPIN 14       // D5 = GPIO14
#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);

//------------ Buzzer ------------------
#define BUZZER_PIN 12   // D6 = GPIO12

void setup() {

  Serial.begin(115200);
  delay(1000);
```

```
  dht.begin();

  pinMode(BUZZER_PIN, OUTPUT);
  digitalWrite(BUZZER_PIN, LOW);

  // Connect WiFi
  Serial.print("Connecting to WiFi");

  WiFi.begin(WIFI_SSID, WIFI_PASS);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println();
  Serial.println("WiFi Connected!");
  Serial.print("IP Address: ");
  Serial.println(WiFi.localIP());

  // Start ThingSpeak
  ThingSpeak.begin(client);

  Serial.println("ThingSpeak Ready!");
}

void loop() {
```

```
float temp = dht.readTemperature();
float hum = dht.readHumidity();

// Check DHT
if (isnan(temp) || isnan(hum)) {
  Serial.println("DHT Error");
  delay(2000);
  return;
}

String status;

// Alert conditions
if (temp > 30 || hum > 80) {

  status = "ALERT";
  digitalWrite(BUZZER_PIN, HIGH);

}
else if (temp > 20 || hum > 70) {

  status = "WARNING";
  digitalWrite(BUZZER_PIN, LOW);

}
else {
```

```
  status = "NORMAL";
  digitalWrite(BUZZER_PIN, LOW);
}

// Serial Monitor
Serial.println("------------------------");

Serial.print("Temperature: ");
Serial.print(temp);
Serial.println(" C");

Serial.print("Humidity: ");
Serial.print(hum);
Serial.println(" %");

Serial.print("Status: ");
Serial.println(status);

// ThingSpeak fields
ThingSpeak.setField(1, temp);
ThingSpeak.setField(2, hum);

// Status as numeric value
// 0 = NORMAL
// 1 = WARNING
// 2 = ALERT
```

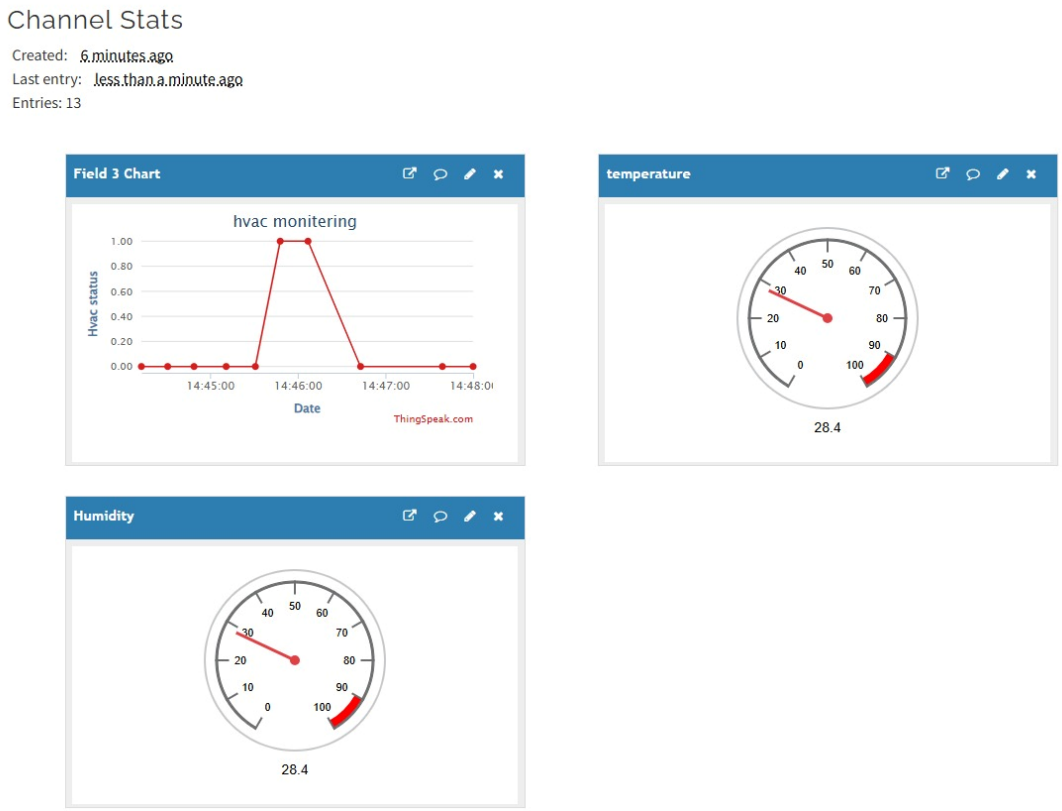
```
  if (status == "NORMAL") {
    ThingSpeak.setField(3, 0);
  }
  else if (status == "WARNING") {
    ThingSpeak.setField(3, 1);
  }
  else {
    ThingSpeak.setField(3, 2);
  }

  // Send data
  int response = ThingSpeak.writeFields(CHANNEL_ID, WRITE_API_KEY);

  if (response == 200) {
    Serial.println("ThingSpeak update successful!");
  }
  else {
    Serial.print("ThingSpeak update failed. Error: ");
    Serial.println(response);
  }

  delay(20000);   // ThingSpeak minimum update interval
}
```

9. System Working:



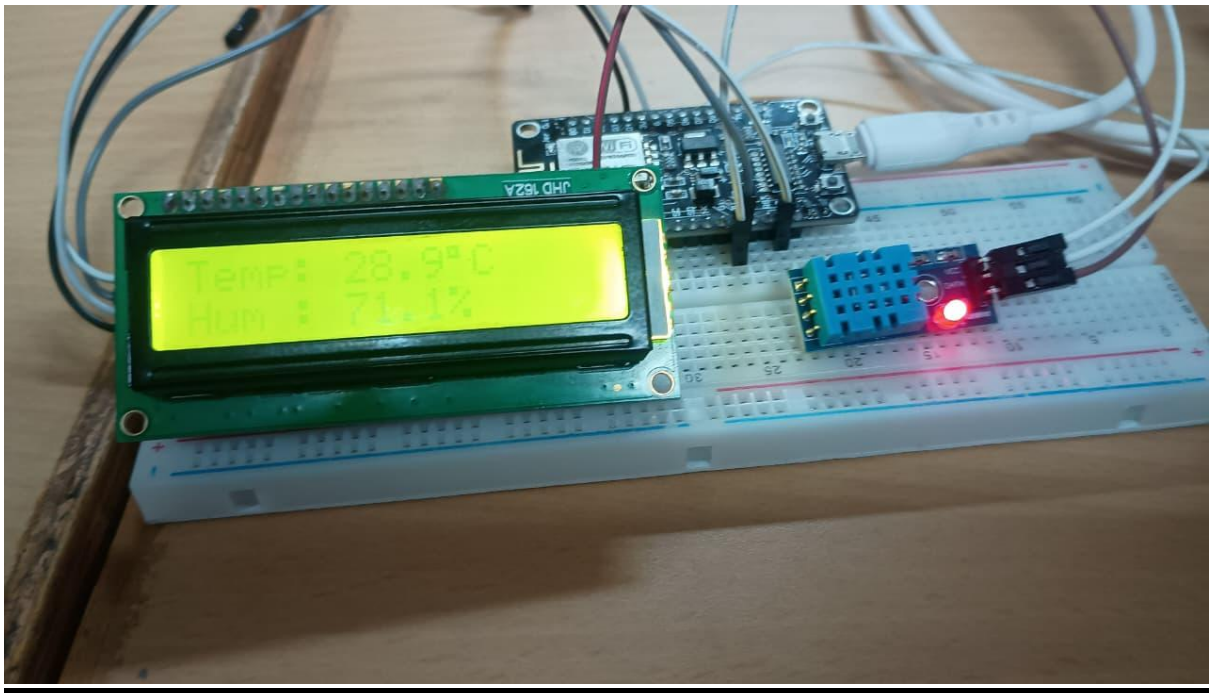
10. Working sequence:

Start → Initialize ESP8266 & DHT11 Sensor → Connect to Wi-Fi → Connect to Cloud Platform → Read Temperature & Humidity → Process Sensor Data → Check Threshold Limits → Display Values → Generate Alert (if required) → Upload Data to Cloud → Remote Monitoring Dashboard → Repeat Process

11. Bill of Materials:

S.No.	Component	Quantity
1	ESP8266 NodeMCU	1 DOCX
2	DHT11 Temperature & Humidity Sensor	1 DOCX
3	OLED/LCD Display	1 DOCX
4	Relay Module	1 DOCX
5	Buzzer	1 DOCX
6	Breadboard	1 DOCX
7	Jumper Wires	1 Set DOCX
8	USB Cable / Power Supply	1 DOCX

12. Prototype Implementation:



13. Results and Performance Analysis:

Performance:

The system provides:

- DHT11 range: 0°C to 50°C (±2°C accuracy) and 20% to 90% RH (±5% RH accuracy).
- Transmission rate: 1 to 5 seconds sampling delay over Wi-Fi.
- Low power consumption via 5V micro-USB supply.
- Real-time local OLED/LCD updates and cloud telemetry publishing.

Advantages:

- Remote real-time monitoring via cloud dashboard.
- Immediate local alert triggering (relay and buzzer).
- Low-cost and readily available hardware.
- Compact and modular design.

Limitations:

- Lower sensor precision and range compared to industrial sensors.
- Requires continuous Wi-Fi connection for cloud updates.
- Lack of environmental enclosure protection.